

Apprentissage machine

L'apprentissage par renforcement

Mickaël Bettinelli
(mickael.bettinelli@univ-smb.fr)



Vue d'ensemble

- Définitions et cadre applicatif
- Aperçu des algorithmes
- Equation de Bellman
- QLearning
- Deep QLearning
- Policy gradient



Attention, nous allons parler franglais !

Apprentissage par renforcement = Reinforcement Learning (***RL***)



RL vs Apprentissage supervisé

- Pas de supervision de l'entraînement
- Pas de données collectées au préalable

Apprentissage supervisé :

- Classification
- Régression

RL :

- Contrôle
- Prise de décision

Exemples d'utilisation du RL :

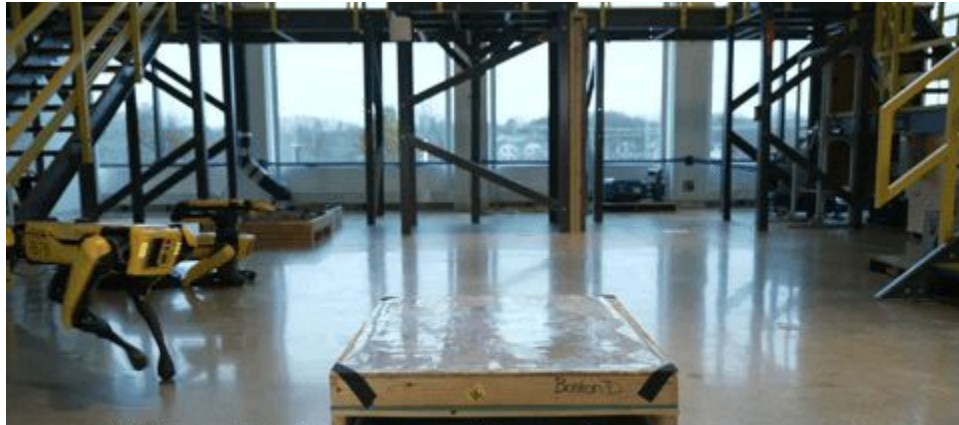
contrôler un robot, jouer à un jeu vidéo, trading en bourse



Analogie à l'être humain

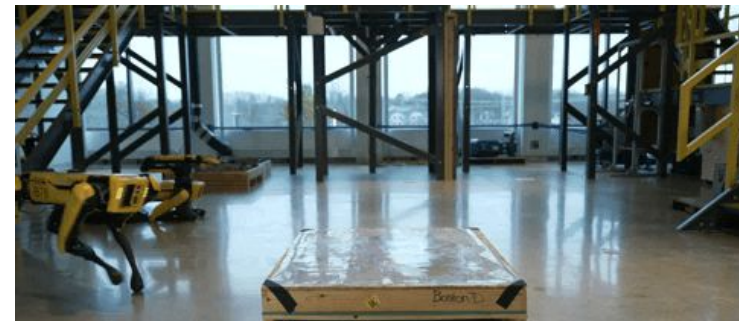
Le RL s'inspire de notre façon d'apprendre :

- Par essais-erreurs





Analogie à l'être humain



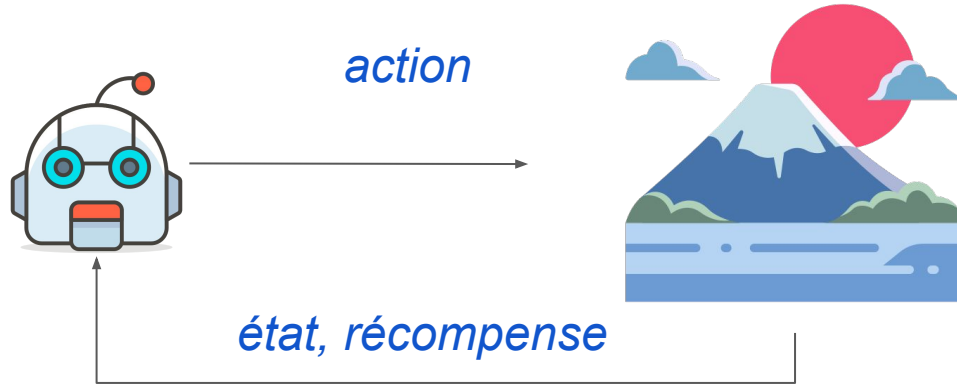
Après chaque action le robot reçoit un **feedback** :

- Si le feedback est **négatif** (le robot est tombé) alors il apprend que sa dernière action était mauvaise.
- Si le feedback est **positif** (il passe) alors il apprend que sa dernière action était bonne.

Le robot a pour objectif de **maximiser les feedback positifs**.



Analogie à l'être humain

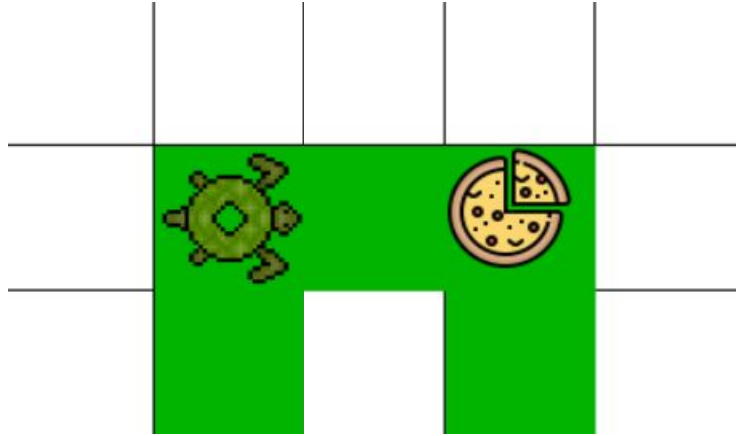


Glossaire :

- *Agent* : une entité capable de percevoir et d'interagir dans son environnement.
- *Etat* : état de l'environnement et de l'agent à un instant donné
- *Action* : action effectuée par un agent sur son environnement
- *Récompense* : récompense donnée à un agent pour avoir atteint un état souhaité (ou non) à la suite d'une action. Une récompense peut être positive ou négative.



Définir un problème de renforcement

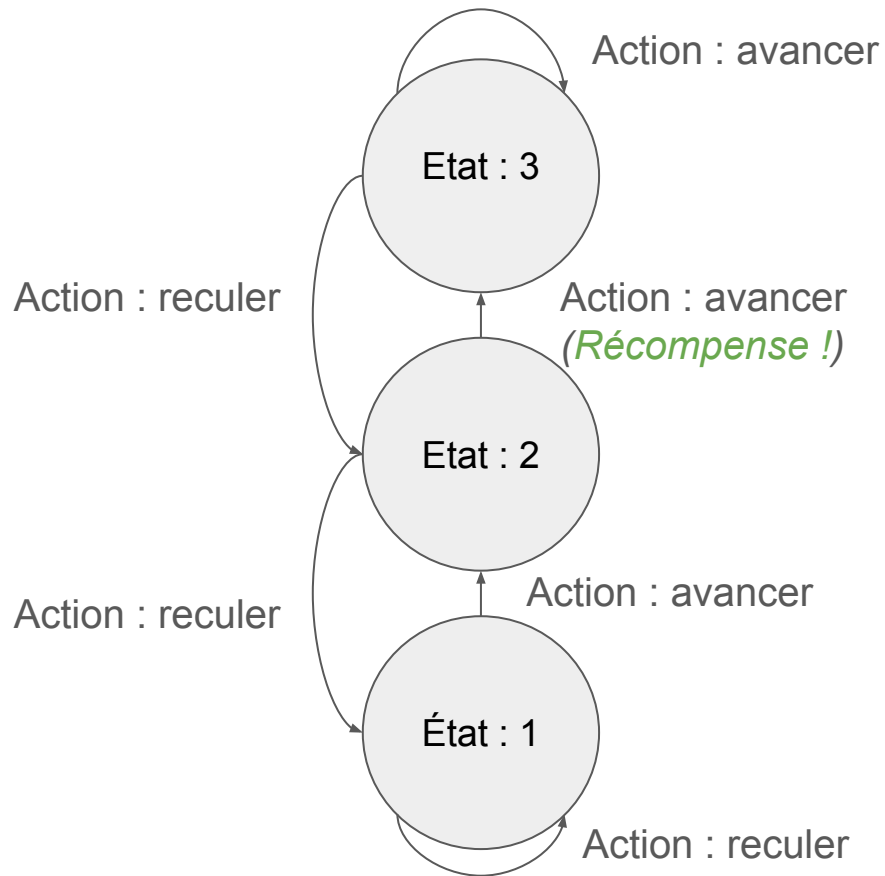
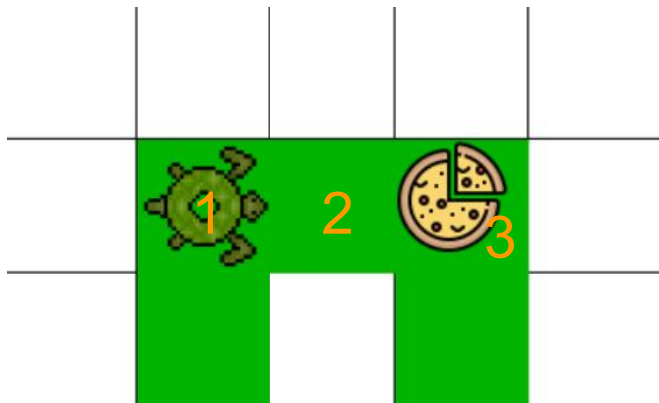


Une tortue est enfermée dans un labyrinthe à la recherche de sa pizza.

Comment modéliser cette situation de prise de décisions ?

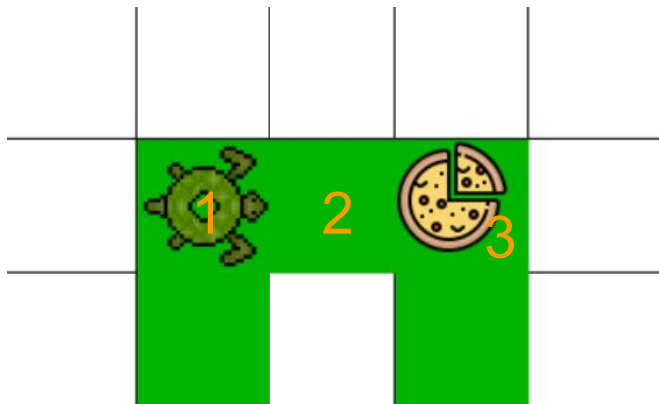


Définir un problème de renforcement



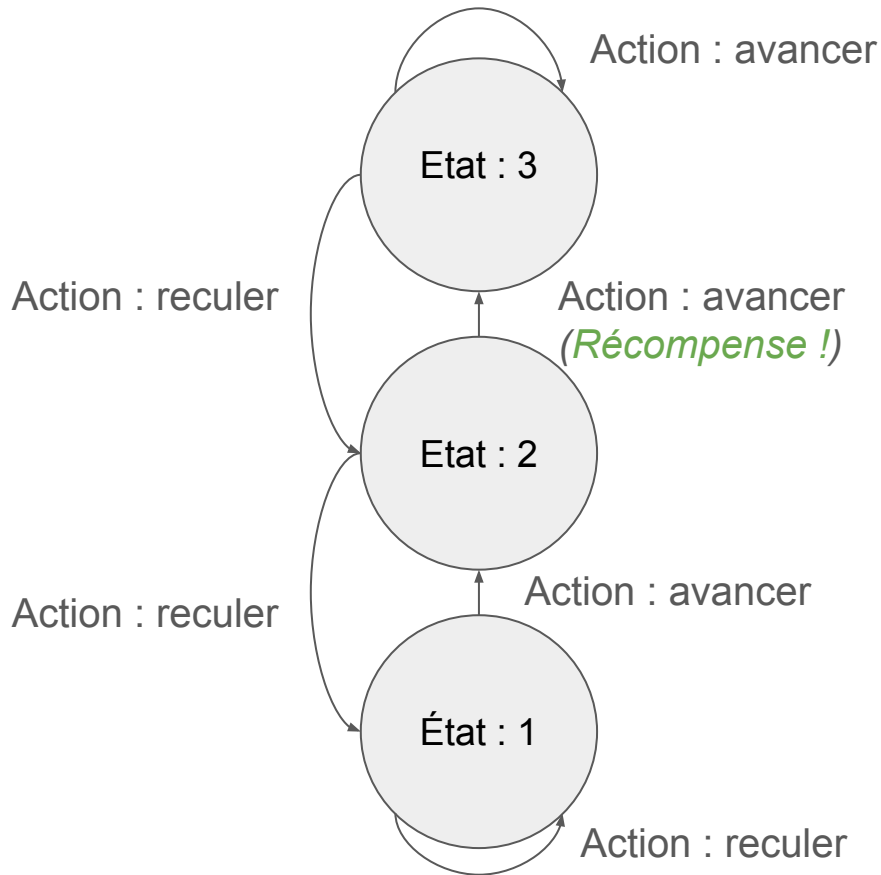


Définir un problème de renforcement



On appelle ça un **MDP** !

(*Markov Decision Process*)

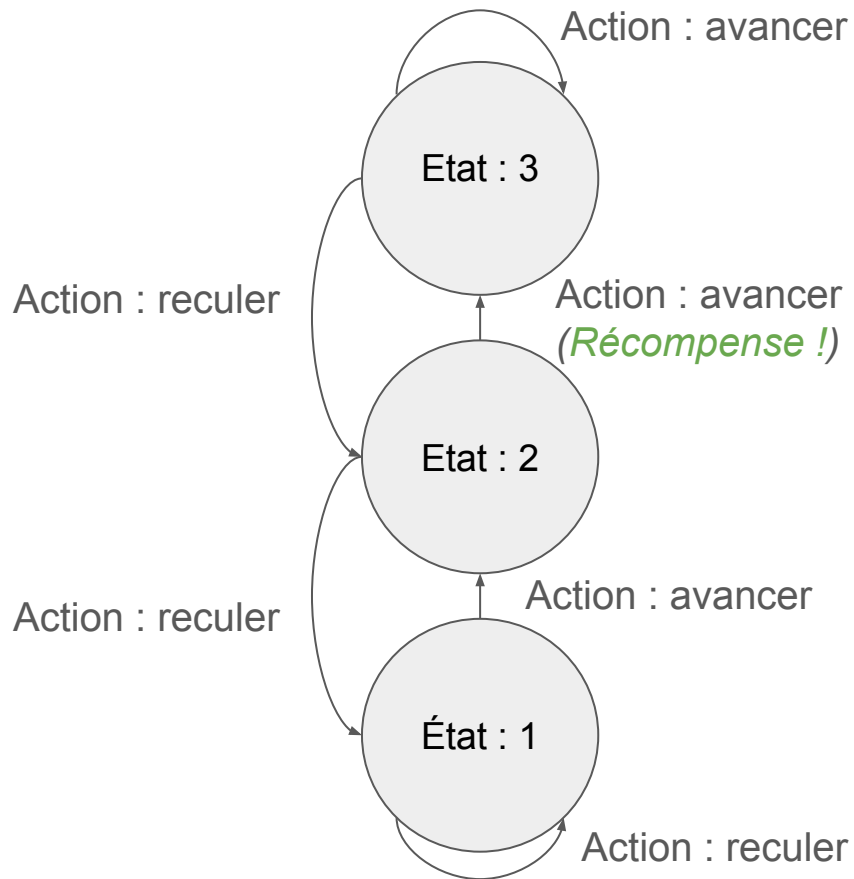




Définir un problème de renforcement

Processus de décision Markovien :

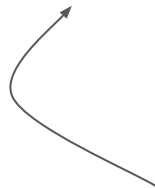
- Processus de contrôle stochastique discret
 - Discret : discrétisation d'un environnement continu sous forme d'état
 - Stochastique : probabilité p que l'action a emmène à l'état s
- $\{S, A, T, R\}$:
 - S : state
 - A : action
 - T : Transition
 - R : reward





Définir un problème de renforcement

- *Objectif de l'agent* : atteindre l'état de récompense avec le moins d'actions possible, en évitant éventuellement les récompenses négatives.
- La stratégie mise en place par l'agent est appelée une **politique (π)**.
- La meilleure politique est appelée **politique optimale (π^*)**.

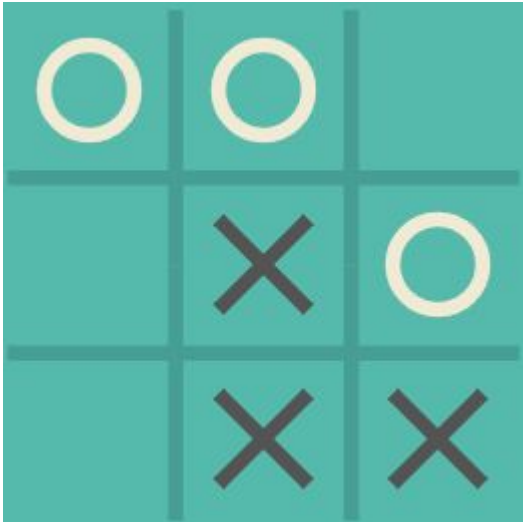


C'est celle que l'on cherche !



Moment exercice

Créer le MDP du jeu Tic Tac Toe à partir de cet état :



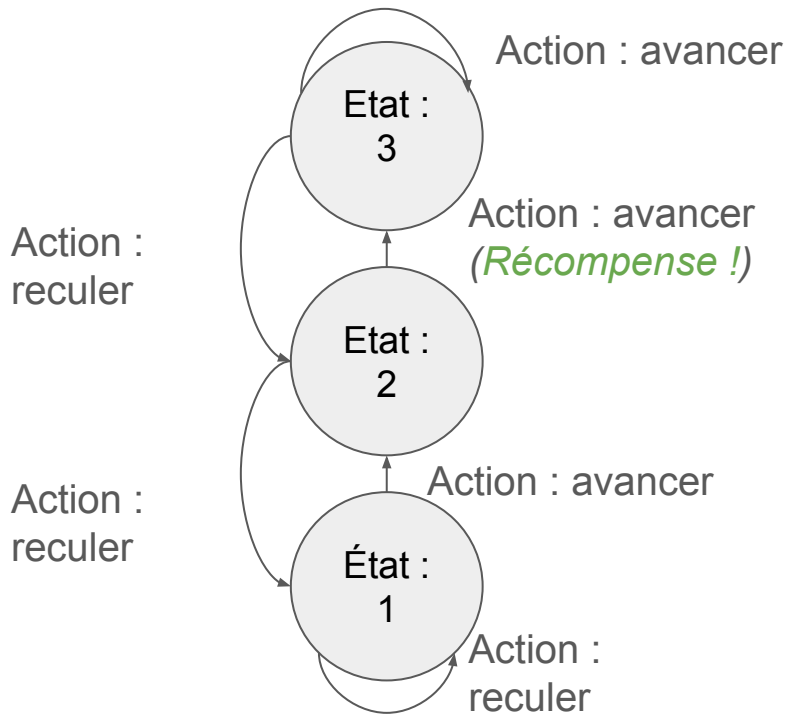
Quelle est la politique optimale pour le joueur *rond* ?

Le prochain joueur doit jouer une croix.



Définir un problème de renforcement

Une autre manière de voir un MDP est sous la forme de tableau, plus pratique pour les développeurs !



Action Etat	a_1 (avancer)	a_2 (reculer)
s_1	$P(a_1 s_1)$	...
s_2	...	...
s_3	...	$P(a_2 s_3)$



Contrôle de l'agent

Les fondements des algorithmes de RL sont le QLearning et SARSA.

La vaste majorité des algorithmes de RL fonctionnent sur le même principe qu'eux. Les comprendre est donc nécessaire !

Les deux utilisent l'***équation de Bellman***, elle aussi reprise dans la majorité du domaine.



QLearning

- Le QLearning permet de remplir notre MDP (appelée QTable).
- QLearning est un algorithme ϵ -greedy
- Plus une valeur V est grande, plus il y a de chance que l'action soit sélectionnée
- La QTable est actualisée après chaque action de l'agent

Comment actualiser la QTable ?

QTable	a_1 (avancer)	a_2 (reculer)
s_1	$V(a_1 s_1)$	...
s_2	...	...
s_3	...	$V(a_2 s_3)$



Equation de Bellman

Objectif de l'actualisation : propager la valeur de la récompense à travers les états

$$Q^{new}(S_t, A_t) \leftarrow (1 - \underbrace{\alpha}_{\text{learning rate}}) \cdot \underbrace{Q(S_t, A_t)}_{\text{current value}} + \underbrace{\alpha}_{\text{learning rate}} \cdot \left(\underbrace{R_{t+1}}_{\text{reward}} + \underbrace{\gamma}_{\text{discount factor}} \cdot \underbrace{\max_a Q(S_{t+1}, a)}_{\text{estimate of optimal future value}} \right)$$

new value (temporal difference target)

- Discount factor : réduire la propagation de la récompense à travers les états
- Learning rate : équilibre la vitesse de propagation des nouvelles valeurs

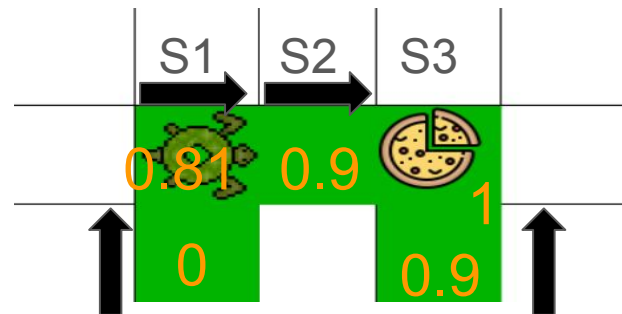
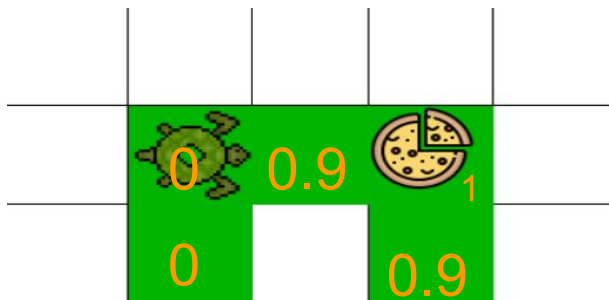
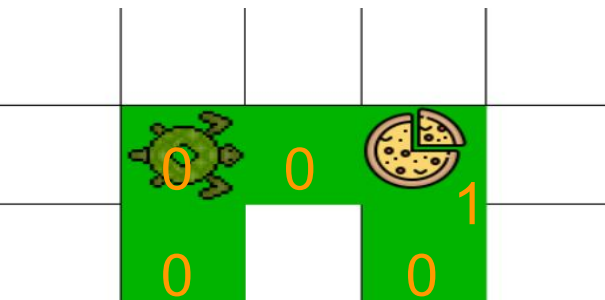


Propagation des valeurs

Règles :

- La case pizza offre une récompense de 1
- $\gamma = 0.9$
- $\alpha = 1.0$
- Transitions déterministes

QTable	Gauche	Droite	Haut	Bas
S_1	0	0.9	0	0
S_2	0.81	1.0	0	0
S_3	0.9	0	0	0.9



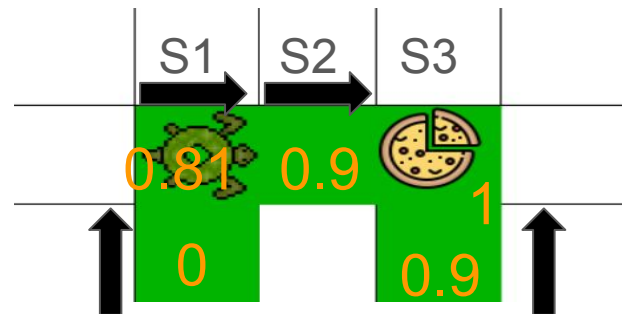
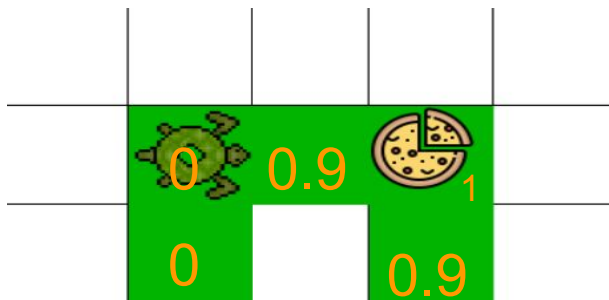
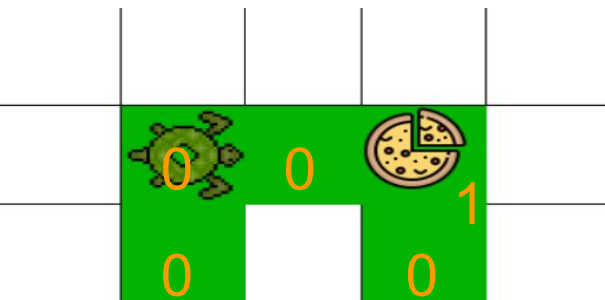


Exploration vs Exploitation

Règles :

- $\epsilon \rightarrow$ (*epsilon*) sélection d'actions aléatoires

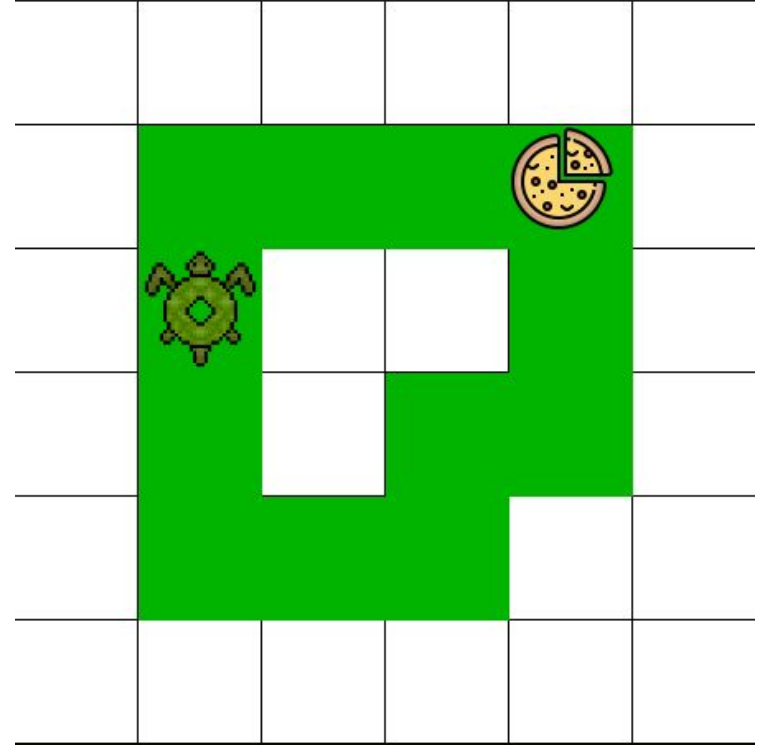
QTable	Gauche	Droite	Haut	Bas
S_1	0	0.9	0	0
S_2	0.81	1.0	0	0
S_3	0.9	0	0	0.9





TP tortue

Développez une tortue qui apprend à se déplacer sur la pizza avec un QLearning



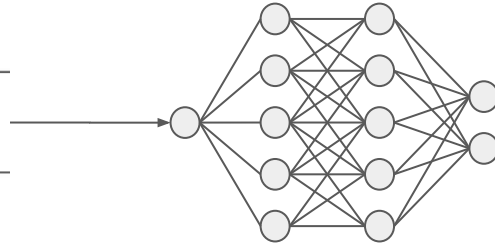
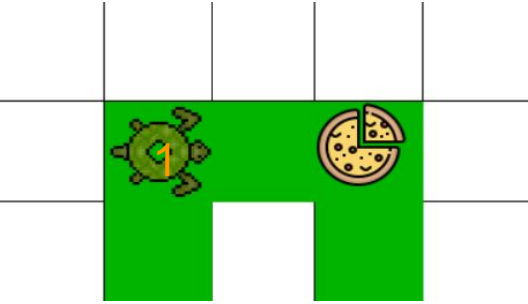


Limite du QLearning

- Pas de passage à l'échelle pour le nombre d'états
- Pas de capacité de généralisation



L'apprentissage supervisé et le RL



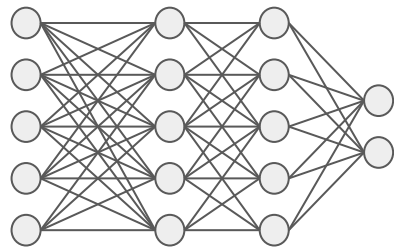
QTable	a_1 (avancer)	a_2 (reculer)
s_1	?	?
s_2	...	...
s_3	...	$V(a_2 s_3)$



Deep Q-Network (DQN)



Policy gradient





Synthèse